

Class #9

03.04 Architecture Development Method (ADM / TOGAF)

Software Architectures · Master in Informatics Engineering

Cláudio Teixeira · claudio@ua.pt

Agenda

- **Block 1 – From Design Passes to Enterprise Change** 10 min

- what changes from ACDM to ADM
- what this cycle must actually produce

- **Block 2 – Worked Example: CampusFlow ADM Cycle** 90 min

- full context and real outputs from Preliminary to H
- what belongs in each phase and artifact

- **Block 3 – Group Exercise: RocketTickets After Acquisitions** 80 min

- Part 1 – Frame, Vision, Business Architecture (30')
- Part 2 – Data, Application and Technology Architecture (25')
- Part 3 – Migration, Governance and Change (25')

Block 1

From Design Passes to Enterprise Change

What makes an ADM problem different

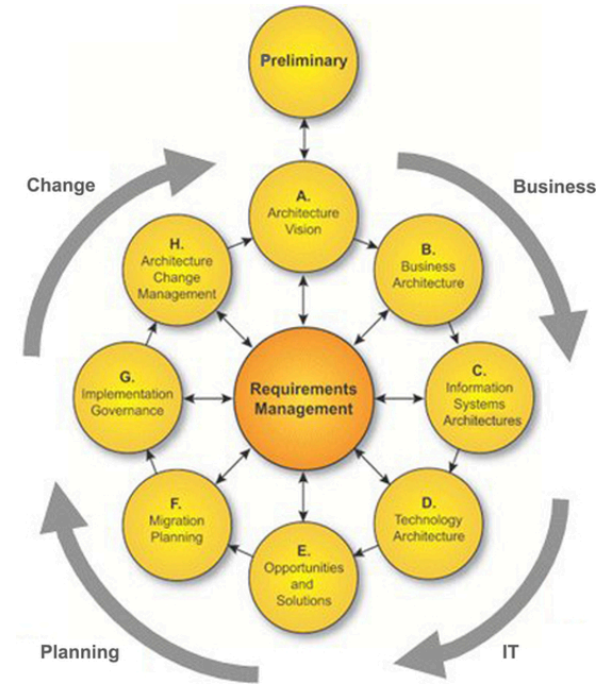
ADM shows up when the architectural challenge is broader than one system design decision.

Challenge	Why it pushes us toward ADM
Enterprise scope	several units, systems, stakeholders, and constraints must be treated together
Business plus IT change	roles, processes, policies, and systems all change at the same time
Baseline to target evolution	we are not only designing a target; we must move from the current estate to it
Coexistence and migration	old and new capabilities often need to live together for a period
Cross-team governance	multiple delivery teams need shared contracts, review points, and exception handling
Controlled change over time	the architecture must survive rollout pressure, deviations, and change requests

So the challenge is not just technical decomposition. It is coordinated enterprise evolution.

ADM cycle – broad view

- this is a cycle, not a straight line followed once and forgotten
- Requirements Management is not its own isolated project phase; it stays active while the cycle moves
- H loops back because implementation always pushes new change back into the architecture
- the outer arrows matter too: business, IT, planning, and change all stay in motion while the cycle runs



Phase Clusters

Plain-language view

1 FRAME THE CYCLE

Preliminary

A – Vision

2 DEFINE THE ARCHITECTURE

B – Business

C – Data + App

D – Technology

4 KEEP ROLLOUT ALIGNED

G – Governance

H – Change Mgmt

3 TURN ARCHITECTURE INTO CHANGE

E – Opportunities

F – Migration

Preliminary + A decides who owns the cycle, what the change is trying to achieve, and which constraints shape it.

B + C + D defines what must change in the business, systems, data responsibilities, and technical environment.

E + F turns the architecture into work packages, sequencing, and coexistence decisions.

G + H keeps delivery aligned while rollout pressure, exceptions, and change requests appear.

What each cluster should leave behind

Phase cluster	Typical output
Preliminary + A	architecture sponsor, principles, stakeholder concerns, cycle vision
B	business capabilities, role changes, capability gaps
C + D	data authority, application responsibilities, integration and runtime guardrails
E + F	work packages, transition slices, migration order, coexistence decisions
G + H	conformance rules, exception handling, controlled change during rollout

The cycle links:

- scope and principles
- business change
- data, application, and technology consequences
- transition planning
- governance and change management

Block 2

Worked Example: CampusFlow ADM Cycle

CampusFlow – full context

CampusFlow is a university-wide student services platform for a group formed by merging three campuses.

Today, each campus still runs its own combination of:

- admissions portal
- enrollment system
- billing and payment workflows
- timetable tools
- LMS integration
- student support channels

The rectorate wants one digital student journey before the next academic year, but the current reality is fragmented and politically sensitive.

Supporting working documents

Published folder

CampusFlow ADM worked documents

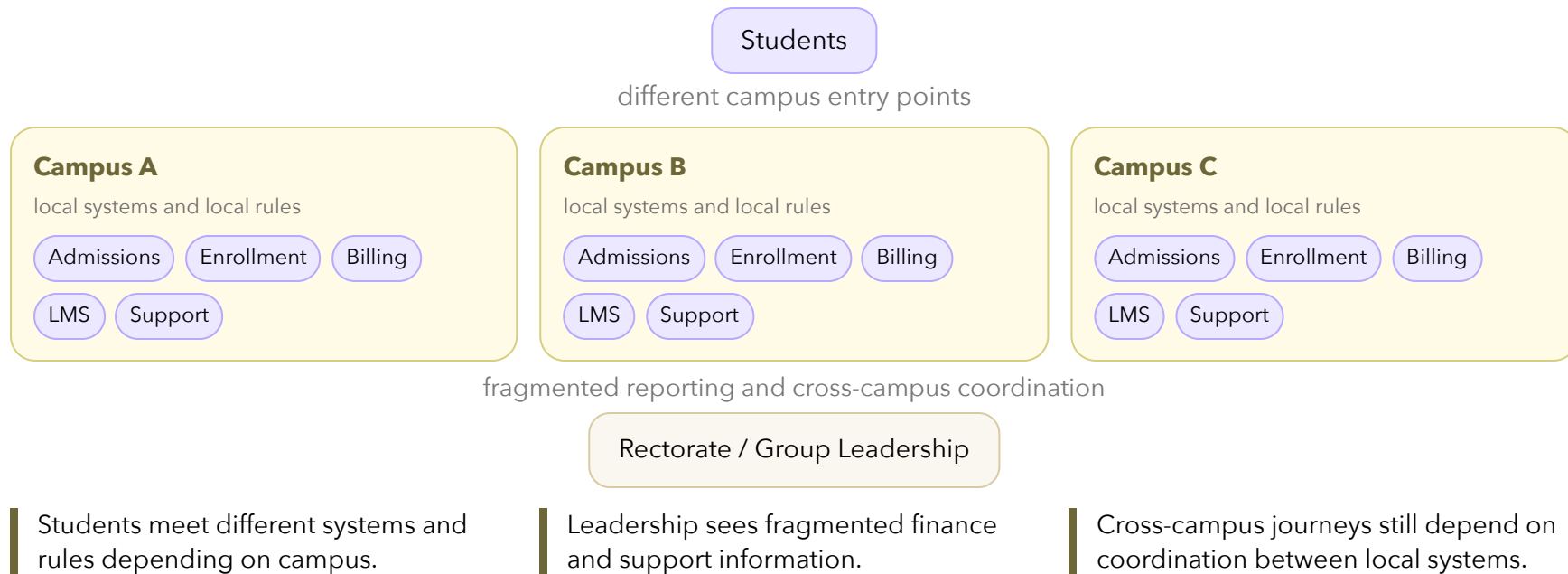
This folder contains the published PDF working documents for **Preliminary** and **Phases A–H** that support the example used in this lecture.

How to use them

- open the folder when you want the full worked pack
- phase entry slides include a bottom-right link to the matching supporting PDF
- the documents read as working ADM outputs for the CampusFlow case

CampusFlow Today

Fragmented baseline



Business drivers and constraints

Business drivers

- one student identity across the group
- one cross-campus support experience
- one finance and billing view for leadership
- easier cross-campus enrollment and timetable coordination

Constraints

- no enrollment-week downtime, meaning no switchover during the live registration week
- the existing LMS stays in place for the first year
- the legacy billing engine cannot be replaced before September
- GDPR and academic-record retention rules still apply

Preliminary – what belongs here

Typical inputs

- executive sponsorship from the rectorate or CIO
- inventory of current systems and owners
- institutional constraints
- political and operational pain points

Questions answered here

- who owns the architecture effort
- who must review or approve major choices
- what principles cannot be violated
- where models, contracts, and decisions will live

Preliminary – concrete outputs

Artifact	Example for CampusFlow
EA team	CIO sponsor, student-services lead, finance lead, DPO, enterprise architect, campus IT reps
Principles	no enrollment-week cutover; one student identity should become authoritative; local LMS may remain temporarily
Repository / tools	one shared place for models, decisions, contracts, and exceptions
Governance structure	architecture steering group plus time-bounded exception process

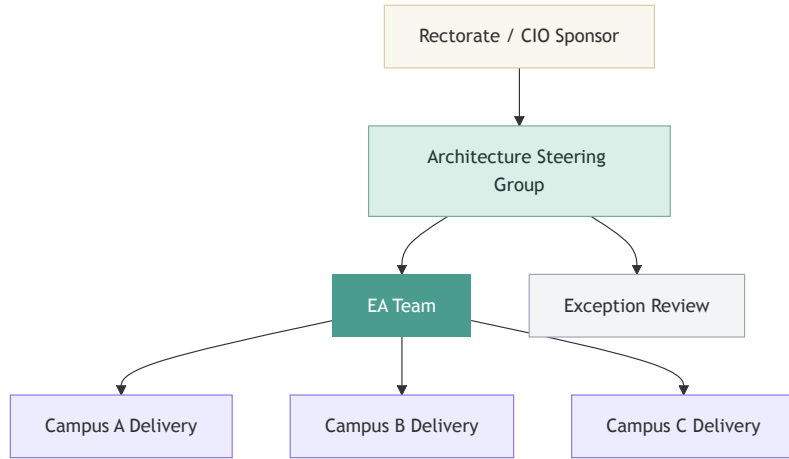
- enrollment-week cutover : moving the live enrollment path from old to new during the busiest registration week
- authoritative identity : the student record trusted when campus records disagree

Preliminary – baseline estate posture

System family	What exists today	Initial posture in this cycle
Enrollment systems	one per campus	retain for execution, integrate and standardize publication
Billing engines	one per campus	retain for execution, expose shared finance visibility
LMS platforms	campus-specific setups	retain for first year, federate access first
Support tools	local inboxes and case histories	migrate toward shared workspace
Timetable tools	local publication logic	keep local execution, standardize publication

- this is where the ADM cycle stops pretending the current estate does not exist
- if you do not name existing systems here, later phases drift into green-field fantasy

Preliminary – governance structure example



- sponsor gives political backing
- steering group resolves conflicts
- EA team shapes the cycle outputs
- exceptions are explicit, not invisible

Phase A – what goes in

Inputs

- business drivers
- key stakeholder concerns
- cycle constraints
- high-level scope questions

Typical stakeholder concerns

Stakeholder	Main concern
Students	one identity and less confusion across campuses
Student services	fewer manual handoffs and clearer case history
Finance	one view of billing and payments
IT operations	no risky cutover during enrollment
DPO / compliance	lawful data sharing and clear ownership

Phase A – what comes out

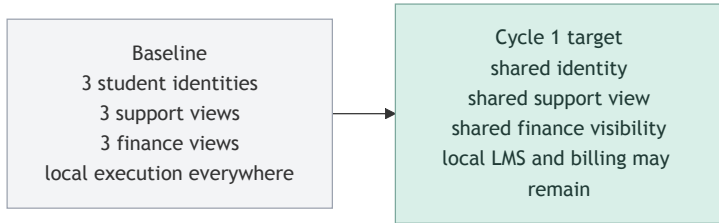
Architecture Vision
Business outcome this cycle:
In scope:
Out of scope:
Main constraints:
Success measures:

Example vision statement:

Before the next academic year, CampusFlow should provide one student identity, one support journey, and one finance visibility layer across the merged campuses, while allowing local LMS and billing execution to remain temporarily in place where replacement risk is too high.

- **finance visibility layer** : a shared consolidated view for finance and leadership, even if billing execution stays local

Phase A – baseline to cycle target



Success measures for the cycle:

- support can resolve a student case without opening multiple campus systems
- finance can see one consolidated billing view
- enrollment week does not depend on a big-bang replacement

Phase A – explicit treatment of existing systems

Existing systems	Decision this cycle	Why
Campus LMS	keep in place	replacement risk is too high before next academic year
Campus billing engines	keep execution local	no safe replacement window before September
Campus enrollment systems	keep in place, add orchestration/integration	continuity matters more than immediate replacement
Campus support tools	start migration to shared workspace	pain is high and benefits are visible
Campus timetable tools	keep local execution, standardize publication	local complexity remains real

- phase A is already saying what will stay, what will move, and what will wait

Phase B – what goes in

Typical inputs

- admissions and enrollment processes
- billing and refund workflow
- student support escalation flow
- ownership of timetable publication

Main business question

Which capabilities need to become group-wide first, and which can remain campus-local for one more cycle?

Phase B – current cross-campus enrollment pain



- `manual handoff` : the request leaves one coherent system path and is pushed forward by people, often through email, tickets, spreadsheet updates, or phone calls
- handoffs are manual or loosely coordinated
- support cannot easily explain where the request is blocked
- finance and timetable checks happen in different local sequences

Phase B – example output

Capability	Today	Direction
Student identity	uplicated by campus	group-wide first
Support case handling	split by campus and channel	one shared case view
Billing visibility	separate per campus	one leadership view, local execution may remain
Timetable publication	local tools and rules	coordinate group-wide, keep local edge if needed
LMS access	campus-specific	federated first, convergence later

This phase should also make roles and responsibilities explicit: student services, finance, IT operations, academic departments, data protection, and support.

Phase B – operating model decisions

Become group-wide first	Stay local for now
Identity	LMS runtime
Support visibility	Billing execution
Finance visibility	Some timetable tooling
Cross-campus policy coordination	Campus-specific operational exceptions

Not every capability centralizes at the same speed. That is one of the main reasons this is an ADM problem.

Phase B – capability gap summary

Capability area	Current gap	Why later phases care
Student identity	no single trusted identity across campuses	shared services and access control depend on it
Support handling	no shared case history or escalation model	support workspace needs a real business target
Finance visibility	no reconciled group-wide view	data publication and reporting must close that gap
Cross-campus enrollment	handoffs are manual and status is opaque	orchestration and integration work depend on this gap
Timetable coordination	publication rules vary by campus	shared contracts and publication paths are needed

- this is the business-side gap picture that later feeds **C** , **D** , and packaging in **E**

Phase C – data architecture inputs

What should be visible here

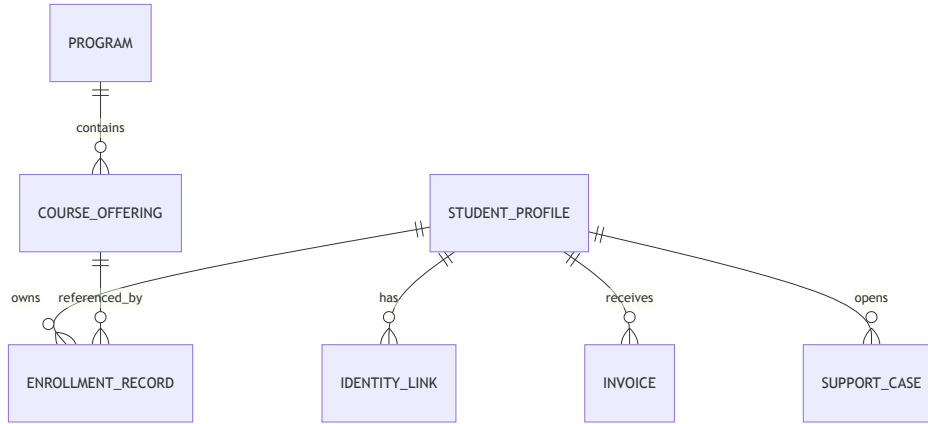
- major entities
- who owns the master record
- sharing and retention rules
- data-quality risks

Typical concerns in this case:

- duplicate identity
- stale timetable data
- billing mismatch
- unclear ownership of support history

Phase C

Data architecture example



Core entities:

Student Profile , Identity Link , Enrollment Record ,
Program , Course Offering , Invoice , and Support Case .

This view clarifies which records behave as shared masters, which stay campus-owned, and which appear as downstream operational records.

Phase C – data ownership and policy outputs

Data concern	Example decision
Student identity	becomes a shared master capability
Enrollment records	campus-driven, but published in standardized form
Billing execution	remains local first
Billing reporting	becomes group-wide
Support history	unified case and timeline visibility

Typical policy outputs:

- retention rules
- sharing rules
- correction workflow
- auditability expectations

Phase C – application architecture inputs

Questions this subphase should answer

- which responsibilities deserve their own boundary
- which consumers need projections instead of direct access
- which interactions must stay away from fragile local runtime paths

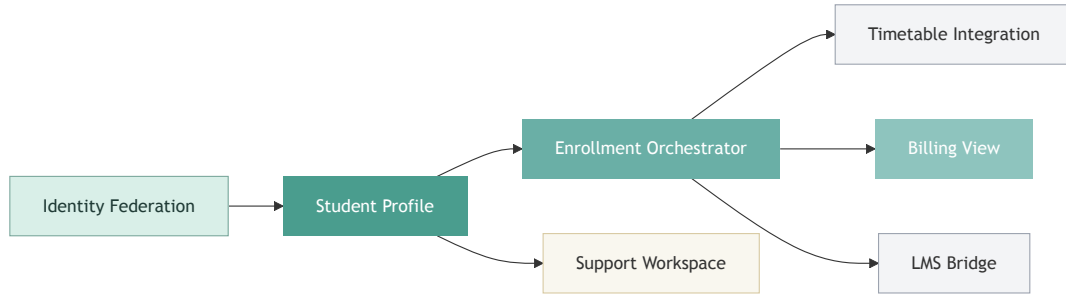
In this case, support, finance, enrollment, and LMS do not all need the same runtime shape.

Phase C – transition treatment of existing systems

Existing systems	Treatment in this cycle	Shared capability they interact with
Campus enrollment systems	remain in place, publish standardized data	Enrollment Orchestrator
Campus billing engines	remain in place, publish finance/reconciliation data	Billing View
Campus LMS platforms	remain in place, consume federated identity	LMS Bridge
Campus timetable tools	remain in place, publish through shared rules	Timetable Integration
Campus support tools	feed or gradually hand over case data	Support Workspace

- phase C is not only inventing new components
- it is also deciding how the new architecture meets the systems that already exist

Phase C – application architecture example



This is one credible application architecture outcome for the cycle.

Phase D – technology architecture inputs

What drives this phase

- the responsibilities and authority decisions from Phase C
- privacy and audit requirements
- peak-period operational constraints
- coexistence with local systems during migration

Technology architecture is where execution rules become explicit.

Phase D – technology outputs

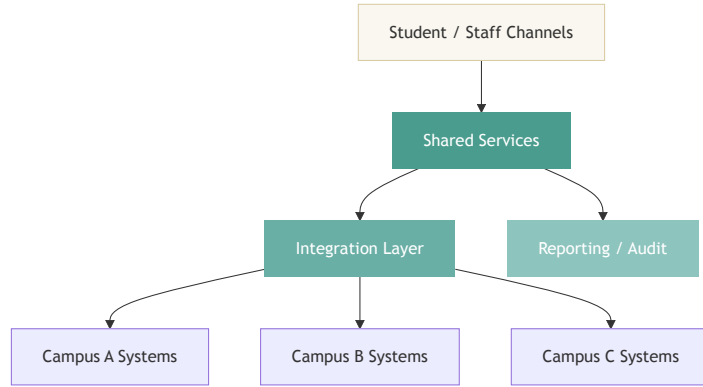
Category	Example for CampusFlow
Software / platform	shared identity provider, integration layer, reporting store, support workspace, adapters/publication paths for campus systems
Runtime constraints	no live dependency on a fragile campus system during enrollment peak
Security / privacy	role-based access, audit logging, GDPR-compliant sharing rules
Standards	event or API contracts for enrollment, timetable, and billing publication
Operations	monitoring and recovery expectations for registration and billing periods

Phase D – inherited technical constraints

Existing system family	Inherited reality	Architectural implication
Enrollment systems	different APIs and timing assumptions	shared journey must hide campus variance
Billing engines	cannot be replaced before September	visibility must be decoupled from execution
LMS platforms	remain live for the first year	identity must federate across multiple LMS runtimes
Support tools	fragmented local data	migration and ingestion rules matter

- phase D is where inherited technical reality becomes architecture rules
- in a migration case, existing runtimes and interfaces are part of the architecture, not background noise

Phase D – runtime zones example



- channels use shared services first
- shared services integrate with campus systems through explicit boundaries
- reporting and audit have their own publication path

Phase E – opportunities and solutions inputs

Inputs

- gap analysis from B , C , and D
- constraints about coexistence and deadlines
- delivery capacity and organizational readiness

Main question

How should the architecture be packaged into programs, projects, or work packages that the university can actually execute?

Phase E – work packages example

1. shared identity and account linking
2. unified student support workspace
3. enrollment and timetable integration layer
4. consolidated finance and billing view
5. later LMS convergence initiative

These are not only “features”. They are delivery vehicles for the architecture.

Phase E – work packages against the existing estate

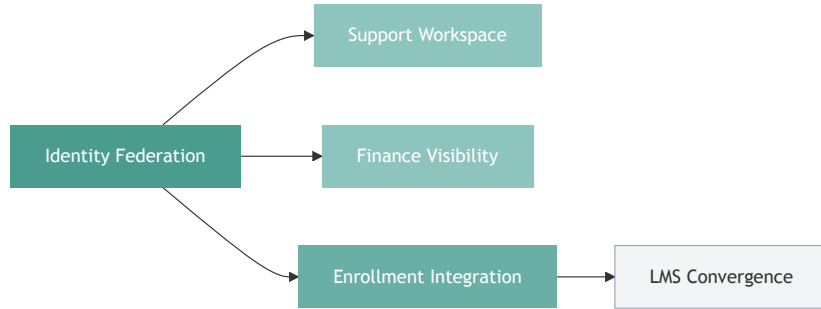
Work package	Which pain it removes	Existing systems touched	What it does not solve yet
Identity federation	duplicate identity and access confusion	campus directories and LMS login rules	LMS replacement
Support workspace	fragmented support handling	campus inboxes and support records	full process unification
Finance visibility	split billing view	campus billing engines and reporting extracts	local billing execution
Enrollment / timetable integration	cross-campus handoffs	campus enrollment systems and timetable tools	all campus tool replacement
LMS convergence	long-term fragmentation	campus LMS platforms	first-year low-risk rollout

Phase E – who actually gets mobilized

Work package	Lead team	Local teams that must design changes
Identity federation	central identity/platform team	campus IAM admins, LMS admins
Support workspace	shared student-services product team	campus support leads and support-tool admins
Finance visibility	reporting/integration team	campus finance systems team and billing owners
Enrollment / timetable integration	orchestration/integration team	campus enrollment and timetable-tool teams
LMS convergence	later academic platforms program	campus LMS teams

- this is where the enterprise architecture starts touching real delivery groups
- phase E names who will actually have to design local changes

Phase E – dependency view



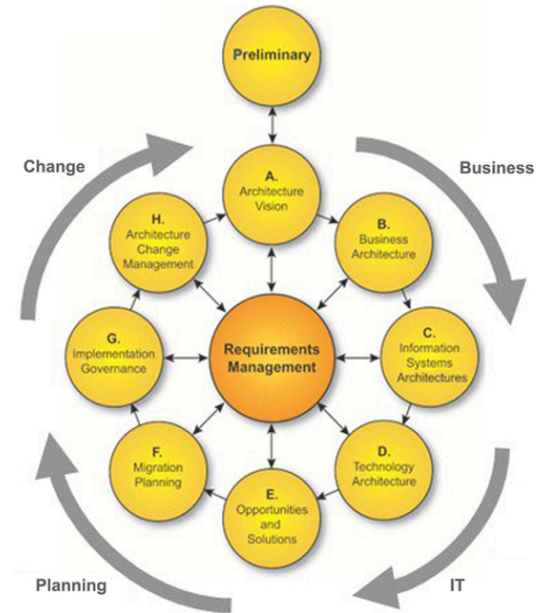
This kind of dependency view is often enough to support the transition discussion.

Phase F – migration planning inputs

This phase should make explicit

- coexistence
- ordering
- risk
- why some local systems survive longer than others

Migration planning is not “implementation detail”. It is part of the architecture.



Helper: in 'E' and 'F' we turn the defined architecture into staged enterprise change.

Phase F – migration example

Stage	Move	Why now	What still coexists
1	identity federation	all other shared views depend on it	local LMS and billing remain
2	support workspace	high visible pain, low operational risk	campus support tools coexist temporarily
3	finance visibility layer	leadership needs one view	local billing execution remains
4	timetable and enrollment integration	improves student journey	campus tools still execute parts locally
5	later LMS convergence	higher risk, lower urgency	federated LMS model persists first

Phase F – disposition of existing systems across the roadmap

Existing system family	During this cycle	Likely later move
Campus enrollment systems	stay as systems of record	replace or consolidate later if justified
Campus billing engines	stay as execution engines	replace later if finance standardization becomes feasible
Campus LMS platforms	stay in place with federated access	converge only after lower-risk capabilities are stable
Campus support tools	coexist briefly with shared workspace	retire local-only handling once migration is complete
Campus timetable tools	stay local but publish through shared rules	rationalize later if variance becomes manageable

Phase F – milestone calendar and responsibilities

Target window	Milestone	Accountable owner	Responsible teams
January	identity and contract baseline approved	enterprise architect	central identity team, local architects, campus IT reps
March	identity federation live for shared services	identity/platform lead	campus IAM admins, LMS admins, operations
April	support workspace live in coexistence mode	student-services product lead	support leads, support-tool admins, DPO
May	finance visibility accepted	reporting/integration lead	finance systems team, billing owners, finance lead
June	enrollment/timetable dry run complete	integration/orchestration lead	enrollment-system teams, timetable-tool owners, academic services
Before enrollment week	protected rollout window approved	enterprise architect	ops lead, campus IT reps, work-package leads
Next academic year start	Cycle 1 target state operational	architecture steering group	central platform teams, campus delivery teams, student services, finance

- phase F should make milestones and ownership visible, not only dependency order
- this is where roadmap timing becomes operational commitment

Phase F – when delivery teams engage

Roadmap stage	When local teams engage	What they prepare
Before Stage 1	after identity package approval	campus IAM and LMS changes for federation
Before Stage 2	after support migration path is agreed	case migration and coexistence design
Before Stage 3	after finance contracts are drafted	billing extracts and reconciliation publication
Before Stage 4	after sequencing is fixed	local adapters and academic-calendar-safe rollout plans

- phase F turns team involvement into actual timing
- teams know when they will be touched and what local design work they owe

Phase F – migration roadmap view



Phase G – implementation governance inputs

What needs governance

- shared identity events
- enrollment publication contracts
- finance visibility and reconciliation
- support case-history completeness
- exception handling

Without explicit architecture contracts, “governance” becomes vague and performative.

Phase G – checkpoint examples

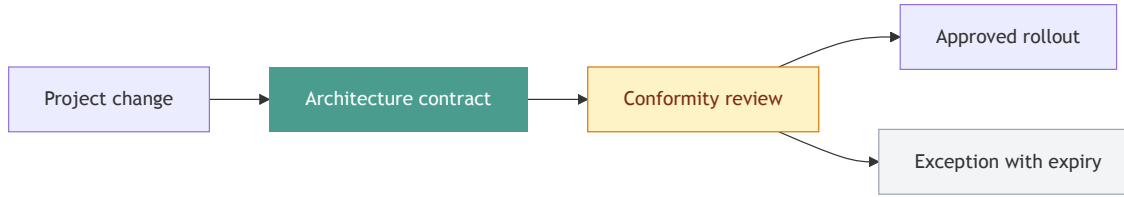
Change area	Example checkpoint
Identity events	conformance review before campus rollout
Enrollment publication	schema and timing review before academic calendar deadlines
Billing visibility	finance reconciliation checkpoint
Support workspace integration	case-history completeness review
Exceptions	time-bounded waiver with owner and expiry

Phase G – when governance touches actual teams

Moment	Teams involved	What governance asks from them
At mobilization	program lead, local system owners, local architects	confirm ownership and local design scope
Before detailed design	local solution/design teams	align on contracts, constraints, and review points
During design	local architects plus shared-platform teams	bring interface and local architecture changes to review
Before rollout	delivery lead, ops, enterprise architect	prove conformance and rollout readiness
If they must deviate	affected team plus governance owner	request a time-bounded exception

- formal governance is not “after teams finish”
- it starts before detailed local design is locked in and continues through rollout

Phase G – governance flow



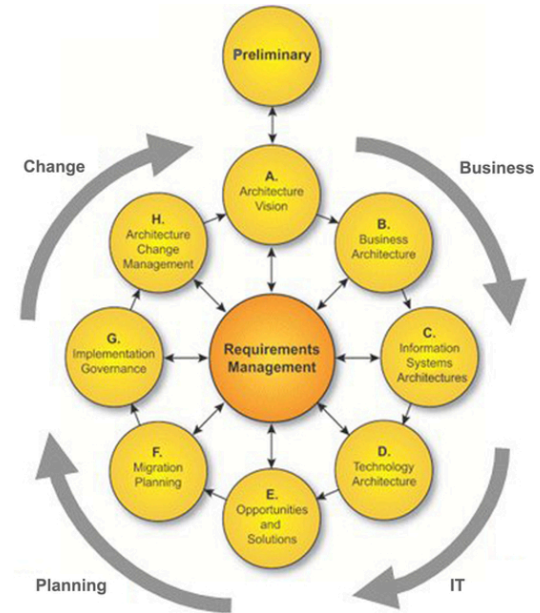
Phase H – change management inputs

Signals that matter

- change requests from delivery teams and local architects
- feedback from students, academic services, finance, and support
- operational incidents
- unresolved exceptions and migration issues
- value, performance, usage, and capacity findings

Phase H maintains the implemented architecture after rollout starts.

It is about value realization, governed change, readiness, and communication, not only about deciding whether to start another ADM cycle.



Helper: in 'H' the cycle absorbs governed change and decides whether refinement stays inside the current vision.

Phase H – what these inputs actually mean

Input	What it means	Example
change request	a team asks to adjust a contract, boundary, rollout step, or local design	billing team needs one more settlement field in the shared publication contract
operational incident	a real failure or near miss reveals a hidden dependency or unsafe assumption	enrollment integration slows down during registration because a campus system is unavailable
unexpected discovery	delivery exposes information we did not know earlier	one campus support records do not have the identifiers needed for a shared timeline
need for refinement	the target still stands, but the architecture must become more precise	one migration step must be split into two stages to keep the rollout safe
readiness or communication gap	the target still stands, but affected teams are not yet aligned to operate it safely	finance visibility is ready, but local finance teams have not yet received the fallback procedure and new responsibilities

- phase H is also where we refine the architecture under the current vision
- not every surprise means “start over”, and not every approved change is purely architectural

Phase H – what it is really doing here

In-flight change	Typical response
new field in an existing publication contract	assess impact and version the contract
rollout window must move because of academic-calendar risk	update migration sequencing
local team cannot yet meet target contract	exception with expiry or staged compliance
architecture is too vague once teams start detailed design	refine the boundary, contract, or guardrail under the current cycle
teams are not yet ready to operate the approved change safely	pause rollout, complete communication and readiness actions, then resume
requested shortcut would reintroduce fragmentation	reject or redesign it
major new constraint breaks the target	re-baseline or start a new cycle

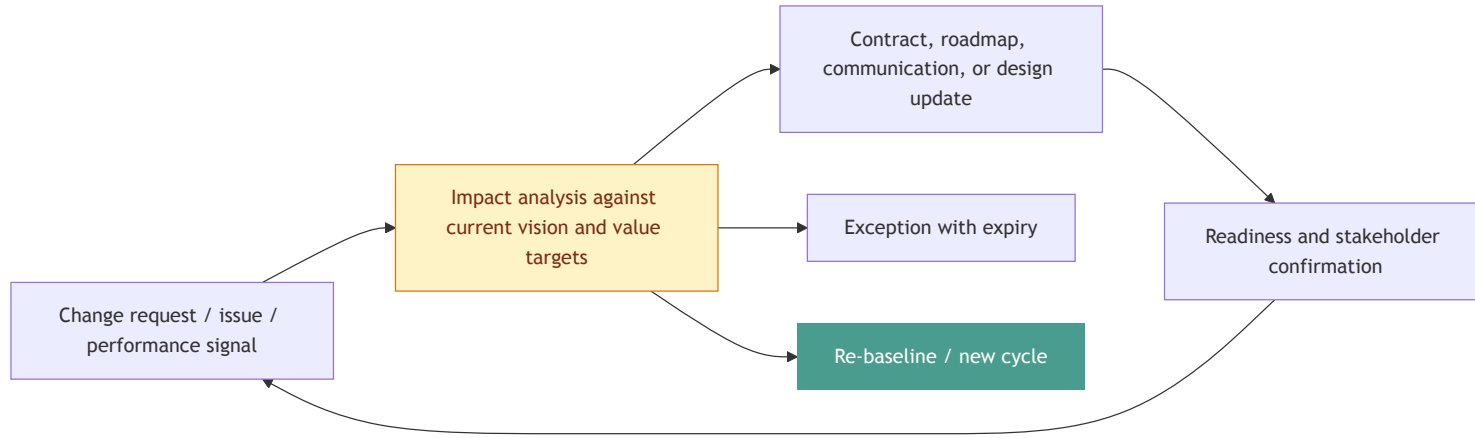
- most Phase H work is about keeping the current target alive under pressure
- only some changes are large enough to reopen the enterprise cycle

Phase H – communication and readiness

Trigger	Stakeholders to align	What must be confirmed
contract update	local architects, delivery leads, system owners	timing, field meaning, version date, rollback impact
rollout-window update	program lead, ops, campus IT reps, business owners	revised milestone, freeze window, fallback procedure
new exception	Architecture Board, local team owner, affected business/platform leads	owner, rationale, expiry, compensating controls
go-live readiness concern	operations, support leads, service owner, enterprise architect	support path, monitoring, escalation, comms pack
value shortfall after release	sponsor, service owner, finance or student-services lead	impact evidence and remediation path

- TOGAF Phase H is also about keeping the change viable in operation
- if stakeholders are not aligned, the architecture is not really ready

Phase H – operating loop



- if the change still fits the vision, Phase H absorbs it under governance
- if it breaks the vision, only then do we escalate beyond the current cycle

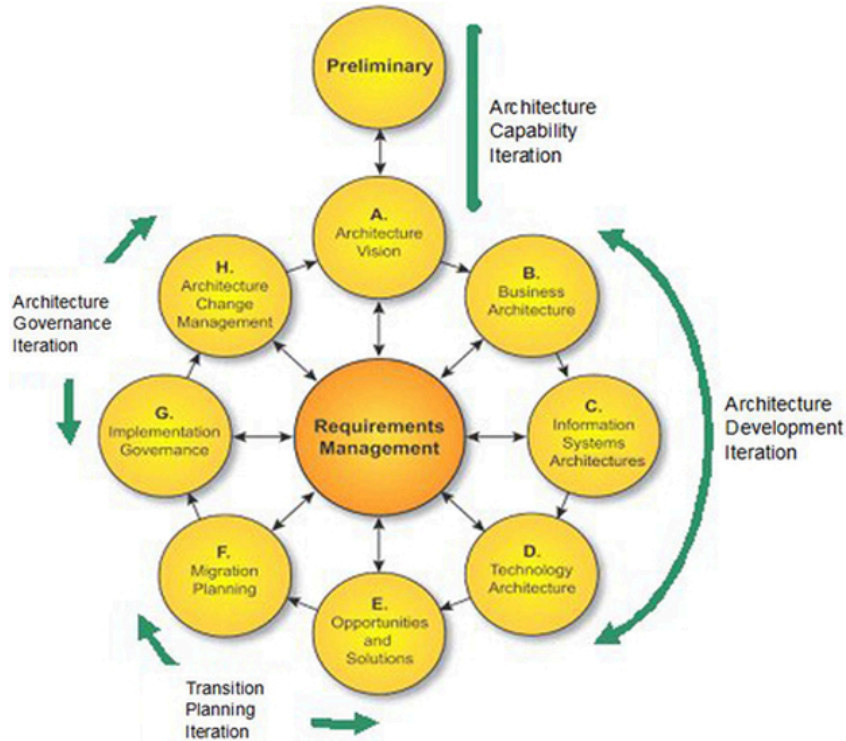
What this worked example leaves behind

An ADM cycle should leave behind:

- a clear architecture vision
- business architecture, not just system architecture
- data and application structure
- technology guardrails
- delivery vehicles and migration sequencing
- governance and change-management mechanisms

That is why ADM is larger than one design pass.

ADM final summary – read it as iterations too



- **Architecture Capability iteration:** prepare or refine how the enterprise does architecture, governance, and sponsorship
- **Architecture Development iteration:** use A to D to define the architecture direction across business, systems, and technology
- **Transition Planning iteration:** use E and F to package, stage, and sequence the move from baseline to target
- **Architecture Governance iteration:** use G and H to keep implementation aligned, manage change, and feed the next cycle
- **Requirements Management** stays active through all of them

Block 3

Group Exercise: RocketTickets After Acquisitions

Part 1 – ADM Frame, Vision, Business Architecture

03.04.01 · Preliminary + A + B · 1:40 → 2:10

RocketTickets bought three companies in 18 months: a regional promoter platform, a venue-operations and gate-validation company, and a resale marketplace. The group wants to operate as one business before next season.

Architecture frame – stakeholders and concerns that must be visible from the start

Architecture vision – what business change this effort exists to enable; what's in/out of scope

Business architecture – group-level capabilities and processes that must change; where cross-company tensions appear



Exercise handout →

30'

GROUP WORK

4

PER GROUP

Part 2 – Data, Application and Technology Architecture

C + D · 2:10 → 2:35

Data and application architecture – group-level bounded areas; data ownership and integration edges that matter most now

Technology architecture – runtime or platform choices forced by the case; standards and guardrails that shape delivery

One focused deep-dive – pick one boundary where ADD-style driver thinking is still needed; write the driver before sketching the response

25'

GROUP WORK

Part 3 – Migration, Governance and Change

E + F + G + H · 2:35 → 3:00

Opportunities and solutions – define the main work packages or transition slices

Migration plan – delivery order; one coexistence problem during transition; which legacy capability survives temporarily and why

Governance – one architecture checkpoint, contract, or conformance rule

Change management – one realistic trigger that would force another ADM cycle

25'

GROUP WORK